ADEPT — TCN Base Learner

Phase 3 Results: Analysis and Next Steps

Guilherme Leitão

Master Thesis — IST / INESC-ID

May 10, 2026

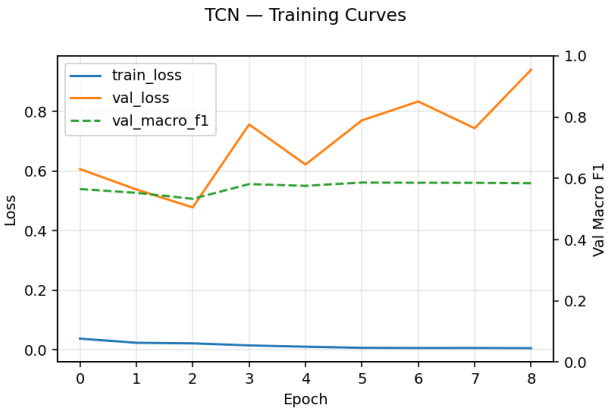
What was trained

- **Task:** same binary classification setting as RF Phase 2 — same 5-attack dataset, same flow-atomic 70/15/15 split (seed=42), same adversarial exclusions (`portscan_adv_*` = 0 windows, `dos_slow` / `syn_dos` / `fuzzing` not on disk).
- **Input:** `sequence.npy` per source — each sample is a $[T{=}10,\ F{=}1090]$ window (1 088 nPrint header bits + IPAT z-score + Direction). Lazy-loaded via memmaps ($\sim$17 GB per source).
- **Architecture:** 3-block causal TCN, channels $[128, 128, 128]$, kernel 3, dilations $\{1, 2, 4\}$, receptive field $29 \gg T$, head: mean+max temporal pooling $\rightarrow$ linear. **806 018 parameters.** AMP enabled (CUDA).
- **Training:** AdamW ($\eta{=}10^{-3}$, wd$= 10^{-4}$), cosine LR schedule, `class_weight=balanced`, early stopping patience 3 on val macro-F1.

Headline metrics (test, overall)

Macro F1	ROC AUC	PR AUC	Recall (adv)	Precision (adv)
0.585	0.707	0.169	0.185	0.266

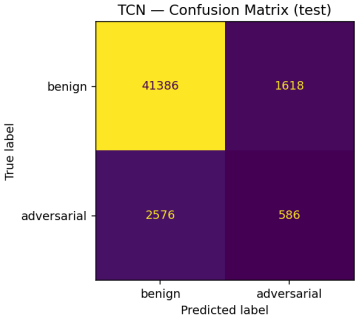
Training behaviour

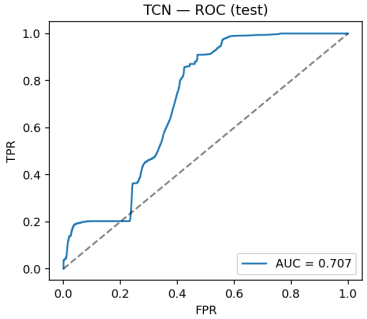


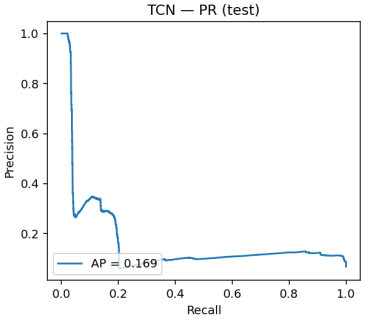
Ep.	Tr. loss	Val. loss	Val. F1
0	0.037	0.607	0.565
1	0.023	0.538	0.553
2	0.021	0.478	0.533
3	0.015	0.756	0.581
4	0.010	0.622	0.576
5	**0.006**	0.770	**0.586**
6	0.006	0.833	0.586
7	0.006	0.743	0.585
8	0.005	0.940	0.584

- **Best checkpoint: epoch 5.** Training loss drops monotonically (model fits the training data) while val macro-F1 stalls in [0.55, 0.59] from epoch 3 onward.
- Val loss oscillates without convergence, suggesting the model is not finding a stable discriminative boundary.
- Early stopping (patience=3) fires after epoch 8 (no improvement at epochs 6, 7, 8).

Confusion matrix & curves







Confusion Matrix | ROC (AUC = 0.707) | Precision–Recall (AP = 0.169)

- **Adversarial recall = 18.5%**: 81.5% of adversarial windows are classified as benign — a severe miss rate.
- **PR-AUC = 0.169**: the precision–recall curve is near the random baseline, indicating very poor adversarial discrimination in probability space.
- **ROC-AUC = 0.707**: better than random — the model has *some* discriminative signal, but the threshold that maximises F1 is far from 0.5 and the curve is not sharp.
- Root cause: Mirai adversarial (79.8% of test adversarial windows) is completely undetected — see next slides.

Per-variant performance (vs. balanced benign)

Variant	Test n	$F1_{adv}$	Recall	ROC AUC
brute_force_adv_both	32	0.783	0.844	0.990
brute_force_adv_iat	32	0.775	0.969	0.987
brute_force_adv_size	32	0.810	1.000	0.995
ddos_adv_both	40	**0.851**	1.000	0.999
ddos_adv_iat	40	**0.105**	**0.075**	0.943
ddos_adv_size	40	**0.860**	1.000	0.992
mirai_adv_both / iat / size	841	**0.000 / 0.000 / 0.000**	**0.000**	0.62–0.67
ssdp_flood_adv_both / size	141	0.829 / 0.817	1.000	0.988
ssdp_flood_adv_iat	141	0.791	0.929	0.969

- **Brute force** (low support, 32 test windows / mode): moderate F1 but interpret cautiously.
- **DDoS IAT**: recall = 7.5% — timing-only perturbations are essentially invisible to the TCN's single IPAT z-score.
- **Mirai: complete failure across all three modes** (F1 = 0.000, recall = 0.000) — 2,523 adversarial windows predicted benign. This single source accounts for 79.8% of all adversarial test windows and drives the poor overall metrics.
- **SSDP Flood**: moderate detection (F1 $\approx$ 0.80–0.83) — weaker than expected given the structural payload changes.

Why Mirai and IAT modes fail: feature geometry

TCN input: 1 090 features per packet

- 1 088 nPrint header bits (IP flags, TTL, ports, TCP flags, payload size bytes, . . .)
- 1 IPAT z-score (*the only timing signal*)
- 1 Direction bit

IPAT share $= 1/1090 \approx \mathbf{0.09\%}$

Consequence for IAT-only variants:
A `_iat` perturbation changes exactly 1 out of 1 090 features per packet. The convolutional filters learn to rely on the 1 088 unchanged header bits, rendering the IPAT z-score invisible.

Why Mirai fails for all modes:
Mirai is a botnet with highly uniform, repetitive traffic. Within a Mirai capture, all packets share essentially the same header structure (fixed SYN/RST pattern, constant payload, fixed port range).
The TCN learns that *any* window with Mirai header patterns is benign (correct for the raw Kitsune traffic). Since TANTGen's perturbations only change IPAT or payload padding — not the header structure — the adversarial Mirai windows look identical to benign Mirai windows from the TCN's perspective. The ROC-AUC of 0.62–0.67 (slightly above random) confirms the model has almost no signal at all.

Contrast with RF: aggregated IPAT statistics (mean/std/min/max) represent 16.7% of the RF input and pick up Mirai's timing shifts robustly (RF F1 $\geq$ 0.953).

RF vs. TCN: head-to-head ($F1_{adv}$, balanced benign)

Variant	RF	TCN	Advantage
brute_force_adv_both / iat / size	0.985 / 0.915 / 0.933	0.783 / 0.775 / 0.810	RF (+0.10 to +0.20)
ddos_adv_both / size	0.988 / 0.975	0.851 / 0.860	RF (+0.11 to +0.14)
ddos_adv_iat	0.509	0.105	RF (+0.40) — both weak
mirai_adv_both / iat / size	0.953 / 0.975 / 0.955	0.000	RF (+0.95 to +0.98)
ssdp_flood_adv_both / iat / size	0.997 / 1.000 / 0.996	0.829 / 0.791 / 0.817	RF (+0.17 to +0.21)
Overall macro-F1	**97.83%**	**58.51%**	**RF (+39.3 pts)**

- **The RF dominates the TCN across every adversarial variant and every metric in this configuration.**
- Unlike the previous 4-attack run (where the TCN won on size/both modes), the 5-attack dataset including brute_force and excluding dos_slow exposes the TCN's inability to handle Mirai and IAT-only perturbations at scale.
- Mirai adversarial windows constitute 79.8% of all adversarial test windows; a complete failure there catastrophically depresses every aggregate metric.

Conclusions

1. **Overall macro-F1 = 58.51%, ROC-AUC = 70.68%** — the TCN fails to discriminate adversarial traffic in the current 5-attack configuration.
2. **Mirai adversarial: complete detection failure (F1 = 0.000) for all three perturbation modes.** Mirai's highly uniform headers leave no structural contrast between benign and adversarial windows in the nPrint space.
3. **DDoS IAT: recall = 7.5%** — IAT-only perturbations are essentially invisible at 0.09% of the input feature space.
4. **Brute force, DDoS both/size, SSDP Flood**: moderate detection (F1 0.78–0.86), but these represent only 20.2% of adversarial test windows, insufficient to lift overall metrics.
5. **The RF + TCN ensemble remains motivated**: the RF covers Mirai and IAT modes where the TCN fails. The meta-learner in Phase 4 should reliably route those cases to the RF signal.
6. **TCN requires architectural revision**: increasing IPAT feature weight, longer window (N=20), or an explicit temporal attention head may recover the Mirai / IAT cases.

Next steps

Short-term — fix TCN and build the ensemble

1. **IPAT reweighting**: augment the IPAT channel with a learned scalar or multi-scale temporal difference features to recover IAT-mode sensitivity without inflating the input dimension.
2. **Window size sweep**: evaluate $N \in \{10, 20\}$; longer windows provide more IAT contrast for slow-jitter attacks.
3. **Phase 4 — Mini-Transformer meta-learner** fusing RF + TCN logit vectors via self-attention. Expected to recover Mirai / IAT variants by routing to the RF when TCN probability is near-uniform.

Long-term — defense layer & write-up

4. **Phase 5 — Purification Transformer** (decoder-only Seq2Seq) projecting adversarial windows back onto the benign manifold; KV-caching for latency.
5. **Phase 6 — RL Adaptation Agent** re-weighting the ensemble online from TANTGen feedback + offline DPI validation.
6. **Dissertation write-up.**

Reproducibility

Artifacts under `ADEPT/data/models/tcn/`

- `tcn.pt` — best model state_dict (epoch 5)
- `metrics.json` — overall + per-source (`vs_all` / `vs_balanced` / `vs_same_base`) + per-label_str
- `split.npz`, `dataset_index.npz` — exact train/val/test partition, shared element-wise with RF via `assemble_index` (`seed=42`)
- `training_info.json`, `training_curves.json`, `tcn_config.json` — full training trace and config
- `*.png` — confusion matrix, ROC, PR, training curves

Re-run from scratch

```
python3 scripts/packet_to_tensor/process_all_attacks.py --tantgen
python3 scripts/tcn/run_tcn.py
```